COMP 206
Intro to Software Systems

Lesson 21: Information Security 1

Recap: networking

- Networked applications organize participants differently
  - Client-server vs peer-to-peer architecture
- Network exists in practice in 4 layers:
  - Link, network, transport, application
- Each layer addresses different concerns:
  - Link: physically transmit data from one device to another with a cable or radio
  - Network: implement packet-switching and route of packets across multiple nodes
  - Transport: identify a particular application on a host, (TCP only:) provide a reliable data stream
  - Application: play games, retrieve web pages, send email, etc.

Recap: TCP socket programming in C

- Both sides: `socket` - configure to use TCP/IP
- Server-side: `bind`, `listen`, `accept`
- Client-side: `connect`

All these functions are **system calls**; we are directly calling OS functions.

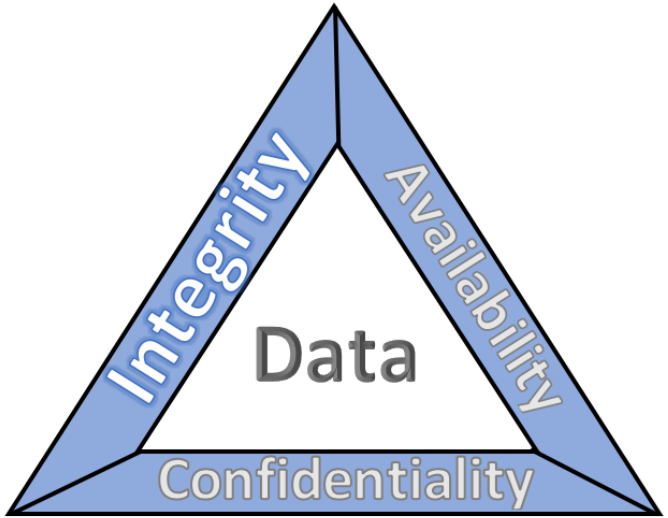
They deal in *file descriptors* (integers) unlike the stdio C library which deals in the more abstract FILE* pointers.

We used `fdopen` to turn a file descriptor into a FILE* so we could use the usual stdio functions to read from and write to sockets.

This week: information security

What is infosec?

- Confidentiality: unauthorized users cannot access information.
- Integrity: information is not lost or corrupted, and is protected from unauthorized modification
- Availability: authorized users can access information.



This is the **CIA triad.** Three pillars of infosec.

Example: email

- Confidentiality:
  - You need to log in to access your emails, perhaps use 2FA.
  - **Encrypted** connection to email server mitigates digital eavesdropping
- Integrity:
  - Email server operators use backed-up systems of distributed storage to avoid data loss
  - **Encrypted** connection to email server mitigates message tampering en route
- Availability
  - Login from anywhere to see your emails

Cryptographic methods: cornerstone of infosec

- Notice both confidentiality and integrity were supported by **encryption** in the example.
- “Cryptography” refers to the practice of ensuring **secure communications** in the presence of **adversarial behaviour.**

Today: cryptography

Next class: offensive security

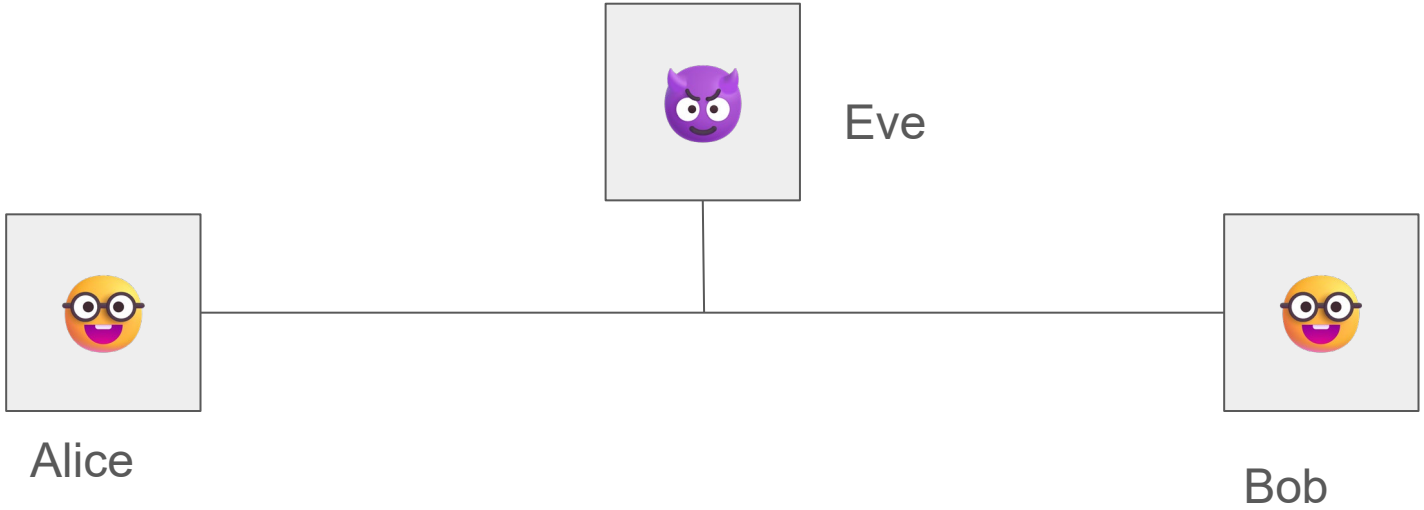
Encryption, decryption, plaintext, ciphertext

- *Encryption* encodes a message (called the *plaintext*), using an *encryption key*, into a form that cannot be decoded (called the *ciphertext*) unless by someone in possession of a *decryption key.*
- *Decryption* takes a ciphertext and its corresponding decryption key to reproduce the initial plaintext.

Intermediary nodes are untrustworthy

This presents an inherent threat to confidentiality and to integrity.

Threat to confidentiality



- If the communication between Alice and Bob is not encrypted, then Eve can learn sensitive information by eavesdropping on the communication.
- Encryption would guarantee that only Bob and Alice could decode the messages and read them.

Threat to integrity

- By inserting herself between Alice and Bob, she can impersonate each of them, tricking Bob into thinking that Alice is saying something that she isn't, and vice versa.
- Cryptographic methods exist to produce *digital signatures*.
- Alice can thus sign her messages, so Bob can check that the message really did in fact come from Alice.

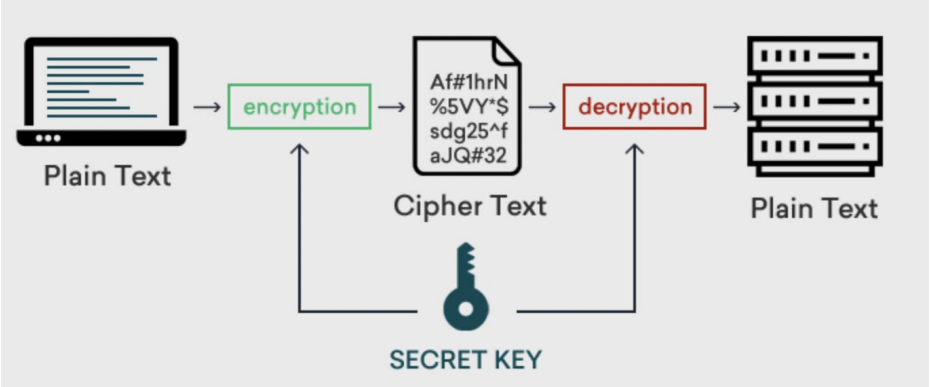
Protecting confidentiality

Symmetric encryption (conceptually very old)

An cryptographic scheme in which the encryption and decryption keys *are the same* is called **symmetric.** Schemes like this have been used for millenia.

Simple example: Caesar cipher (~50 BCE) -- just shift letters over!

Modern example: Advanced Encryption Standard (AES) algorithm (2001)



Caesar cipher

Plaintext: THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG

Ciphertext: QEB NRFZH YOLTK CLU GRJMP LSBO QEB IXWV ALD

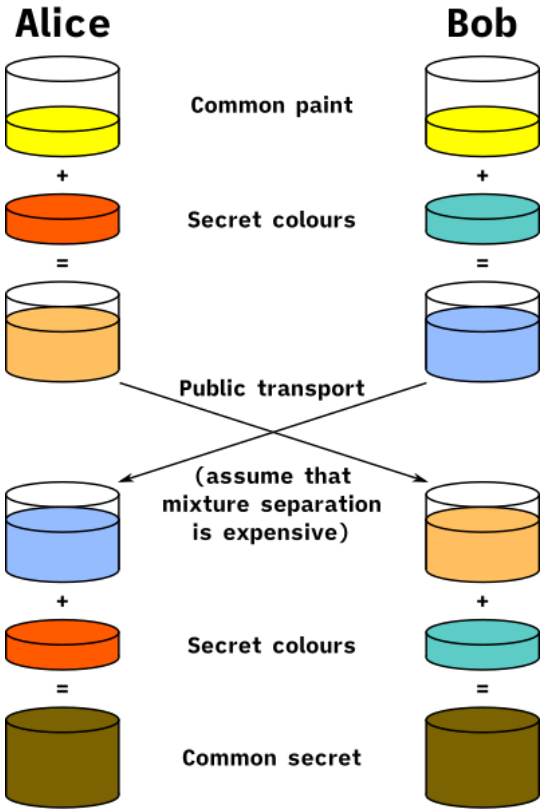
In this example, what is the *shared key*, used to encrypt and decrypt?

Problem: how to agree on a secret key?

Diffie-Hellman key-exchange (1976)

Enables *deriving a shared secret* over an untrusted communication channel, e.g. the Internet.

Relies on **computational infeasibility:** it would take **impossibly long** for an attacker to “separate the paints.”



Separating paints? Number theory

This is not a math class!

D-H needs a mathematical function that is easy to compute, but hard to reverse

- Such functions are called “trapdoor functions”
- Most common example: multiplying large prime numbers
- Toy (insecure) example system if you like math: KidRSA https://macs4200.org/chapters/11/1/kidrsa

Transport Layer Security (TLS)

TLS is a **protocol** that establishes how two parties communicating over TCP negotiate what encryption strategy to use.

TLS can use Diffie-Hellman to establish a shared secret key.

TLS can use AES with that shared secret to protect confidentiality.

But this is insufficient to protect against fraud!

- A client connects to google.com
- An impersonator can intercept the communication and just pretend to be google.com!
- The impersonator still goes through the whole TLS protocol to establish an encrypted channel.
- The client then sends their credentials across the secure channel to the impersonator.

Oops. (This is called a *man-in-the-middle* attack.)

Protecting integrity

Protecting against *accidental* data corruption

- A **checksum** is a calculation done on a message to produce a kind of *summary* that is included alongside the message.
- The receiver of the message performs the same calculation on the message, and compares with the bundled checksum.
- In case of a mismatch, the receiver knows the message was corrupted.

Example: Luhn’s algorithm (credit cards)

- The last digit of a credit card is actually a kind of checksum!
- This is to mitigate **data entry errors** (typos), not fraud.

The algorithm:

- Starting from the first digit, double every other digit in the card number
  - If the result of doubling is longer than 1 digit, add the two digits together
- Sum all the digits together
- If the result is divisible by 10, the card number is valid

The last digit of your CC number is selected so as to make the above algorithm work.

Quick aside: hashing

A *hash function* is a function that turns *any string* into a fixed-size, fairly small integer called a *hash*. Lots of applications, e.g. hash-tables.

Cryptographic hash functions have the following properties desirable for cryptographic purposes.

- Infeasible to find an input string that produces a given hash.
- Infeasible to find another string that produces the same hash as a given string.
- Random input strings produce hashes that are uniformly distributed.

Protecting against forgery: message authentication codes

- A message authentication code (MAC) is essentially an *unforgeable checksum.*
- Using a *pre-shared key*, one can create a MAC by leveraging cryptographic hash functions:
  - Append the preshared key to the end of the message to send.
  - Compute the hash of the resulting augmented message.
  - Send the original message together with the hash.
  - The receiver appends the preshared key to the end of the message, hashes, and compares with the provided hash.
- The receiver can be sure that the sender is authentic thanks to the properties of cryptographic hash functions.

When pre-sharing is infeasible

Asymmetric cryptography (1978)

- These are schemes in which the encryption and decryption keys are *distinct.*
- Rivest, Shamir, and Adleman published the first asymmetric cryptosystem in 1978, called the **RSA algorithm**.

RSA is based on the observation that it is practical to find three whole numbers *n*, *d*, and *e* such that for any “message” *m* where 0 <= *m* < n

$$(m^e)^d \% n == m$$

- *e* is called the **public key**; exponentiation with *e* **encrypts.**
- *d* is called the **private key**; exponentiation with *d* **decrypts.**

It is **computationally infeasible** to compute *d* from *e*, or vice versa.

More number theory

- The computational infeasibility arises from the use of **very large prime numbers** in deriving *e*, *d*, and *n*.
- There are **no known algorithms** that can efficiently find the prime factors of numbers.

Verifying message authenticity with RSA

- Someone desiring to send a message to the owner of a private key can use their public key to send a message that only they can decrypt.
- This is used to create a “challenge” to prove ownership of the private key.
- “I’m google.com” “Okay, prove it! Decrypt this.”
- If response is the correct decryption, respondent is really google.
  - (Note: there could still be a man-in-the-middle, but they can’t be lying about what was said.)

Example: SSH keys

Public Key Infrastructure (PKI)

- Great! Now we just need to get our hands on the public keys of… every web site ever.
- Idea: delegate trust to a few selected authorities, called **certificate authorities.**
- When connecting to google.com, the server presents its public key **together with a message authentication code** signed by a **certificate authority**.
- Your operating system ensures that you have the public keys of the certificate authorities.